



# 网络安全创新创业实践

## 对称密码算法的软件实现

崔鸣洋 20230046168 网安 23.1

2026 年 7 月 26 日

## 1 实验目的

该实验目的为对多种对称密码进行执行效率以及多种软件模式的优化实现

## 2 实验原理

分别通过对每种无优化的算法以及其对应的优化算法进行效率对比，进而得到优化效果的直观体现。

## 3 实验步骤与结果

### 3.1 AES

先完成无优化的AES128代码编写，然后再分别进行T-table / AES-NI / Shuffle三种优化，最后编写代码比较程序运行速率即可

```
gcc -O3 -Wall \
basic/aes.c \
ttable/aes_ttable.c \
aesni/aesni.c \
gcm/gcm.c \
benchmark/bench.c \
-maes -msse4 -mpclmul \
-o aes_benchmark
(base) kobe@DESKTOP-DOKR59F:~/AES_OPT$ ./aes_benchmark
=====
AES Optimization Benchmark
Data size: 100 MB
=====

AES Basic C
Time : 0.7526 s
Speed: 132.87 MB/s

Te0[0]=c66363a5
AES T-table
Time : 0.2477 s
Speed: 403.79 MB/s

AES-NI
Time : 0.0100 s
Speed: 9950.90 MB/s

(base) kobe@DESKTOP-DOKR59F:~/AES_OPT$
```

### 3.2 SM4

完成无优化的SM4代码编写，然后再分别进行T-table / AVX2三种优化，最后编写代码比较程序运行速率即可

```
(base) kobe@DESKTOP-DOKR59F:~/SM4_OPT$ nano src/benchmark.c
(base) kobe@DESKTOP-DOKR59F:~/SM4_OPT$ nano Makefile
(base) kobe@DESKTOP-DOKR59F:~/SM4_OPT$ make clean
rm -rf bin/*
(base) kobe@DESKTOP-DOKR59F:~/SM4_OPT$ make
mkdir -p bin
gcc -O3 -Wall -mavx2 \
src/sm4_basic.c src/sm4_ttable.c src/sm4_ttable_table.c src/sm4_simd.c src/sm4_sbox.c src/benchmark.c \
-Iinclude \
-o bin/sm4_benchmark
(base) kobe@DESKTOP-DOKR59F:~/SM4_OPT$ ./bin/sm4_benchmark
=====
SM4 Optimization Benchmark
Data size:100 MB
=====

SM4 Basic
Time : 0.7124 s
Speed: 140.38 MB/s

SM4 T-table
Time : 0.5229 s
Speed: 191.23 MB/s

SM4 SIMD AVX2
Time : 0.0967 s
Speed: 1033.77 MB/s
(base) kobe@DESKTOP-DOKR59F:~/SM4_OPT$
```

### 3.3 gift

完成无优化的gift代码编写，然后再分别进行bitslice/ AVX / Shuffle三种优化，最后编写代码比较程序运行速率即可

```
wsl
(base) kobe@DESKTOP-DOKR59F:~/gift_basic$ nano src/benchmark.c
(base) kobe@DESKTOP-DOKR59F:~/gift_basic$ make clean
rm -f bin/*
(base) kobe@DESKTOP-DOKR59F:~/gift_basic$ make
gcc -O3 -Wall \
src/gift.c \
src/gift_bitslice.c \
src/gift_avx.c \
src/benchmark.c \
-Iinclude \
-mavx2 -mssse3 \
-o bin/gift_benchmark
(base) kobe@DESKTOP-DOKR59F:~/gift_basic$ ./bin/gift_benchmark
=====
GIFT Optimization Benchmark
=====

Basic GIFT
Time : 7.168689 s
Speed: 2.13 MB/s

Bitslice GIFT
Time : 0.060426 s
Speed: 252.52 MB/s

AVX2 Shuffle GIFT
Time : 0.030079 s
Speed: 1014.58 MB/s
(base) kobe@DESKTOP-DOKR59F:~/gift_basic$
```

### 3.4 TWINE

完成无优化的TWINE代码编写，然后再分别进行bitslice/ SSE两种优化，最后编写代码比较程序运行速率即可

```
wsl
ey.c src/twine_sse.c src/twine_sse_sbox.c src/twine_sse_key.c src/benchmark.c \
-Iinclude \
-o bin/twine_benchmark
(base) kobe@DESKTOP-DOKR59F:~/twine_opt$ ./bin/twine_benchmark
START
=====
TWINE Optimization Benchmark
=====

Data size : 1 MB
Running Basic...
encrypt block = 10000
encrypt block = 20000
encrypt block = 30000
encrypt block = 40000
encrypt block = 50000
encrypt block = 60000
encrypt block = 70000
encrypt block = 80000
encrypt block = 90000
encrypt block = 100000
encrypt block = 110000
encrypt block = 120000
encrypt block = 130000
Running Bitslice...
Running SSE...

-----
TWINE Basic C
Time : 0.962027 s
Speed: 1.04 MB/s

TWINE Bitslice
Time : 0.012990 s
Speed: 76.98 MB/s

TWINE SSE
Time : 0.084807 s
Speed: 11.79 MB/s

-----
Bitslice Speedup : 74.06x
SSE Speedup      : 11.34x
-----
(base) kobe@DESKTOP-DOKR59F:~/twine_opt$
```

## 4 总结与心得

AES:

| 优化方式                      | 优化对象                                                   | 核心技术                                                                          | 优点                                                                           | 缺点                                                                             |
|---------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 基础 C 实现 (Basic AES)       | AES整体加密流程，包括 SubBytes、ShiftRows、MixColumns、AddRoundKey | 按照AES标准算法直接使用C语言实现；通过S-box查表、有限域乘法完成轮函数                                       | ① 实现简单，结构清晰② 易于理解和验证AES算法正确性③ 移植性强，不依赖硬件环境                                   | ① 大量字节级操作，计算效率低② MixColumns涉及大量GF(2 <sup>8</sup> )乘法运算③ CPU并行能力利用不足④ 性能约百MB/s级 |
| T-table优化                 | AES轮函数中的 SubBytes + ShiftRows + MixColumns             | 预计算4个T表 (Te0~Te3)，将多个AES操作合并为32位查表和异或操作                                       | ① 通过空间换时间减少计算量② 将多个轮操作合并，提高软件执行效率③ 对没有AES硬件指令的CPU仍有明显加速效果④ 利用CPU Cache提高访问速度 | ① 占用额外内存（约4KB表空间）② 存在Cache Timing侧信道攻击风险③ 查表访问导致数据相关内存访问④ 在现代CPU上性能低于AES-NI    |
| AES-NI指令优化                | AES轮函数 (SubBytes、ShiftRows、MixColumns、AddRoundKey)     | 利用Intel AES-NI硬件指令：• AESENC• AESENCLAST• AESKEYGENASSIST通过SIMD寄存器(XMM)执行AES运算 | ① 直接调用CPU硬件AES执行单元② 性能提升巨大，可达到GB/s级③ 常数时间执行，避免T-table缓存攻击④ CPU流水线和SIMD并行充分利用 | ① 依赖CPU支持AES-NI指令集② 可移植性降低③ 需要针对不同CPU架构优化④ 调试难度高于纯软件实现                         |
| AES-NI + PCLMULQDQ实现GCM优化 | AES-GCM模式中的CTR加密和GHASH认证计算                             | AES-NI负责CTR模式加密；PCLMULQDQ指令完成GF(2 <sup>128</sup> )无进位乘法，加速GHASH认证             | ① 同时优化加密和认证过程② 大幅提升AES-GCM吞吐率③ 利用硬件完成有限域乘法④ 当前TLS、IPsec等协议广泛采用⑤ 安全性高，无查表泄露风险 | ① 依赖AES-NI和PCLMULQDQ硬件支持② 实现复杂度较高③ 需要处理GHASH约减和数据布局问题④ 调试难度较大                  |
| SIMD并行优化 (补充)             | 多数据块AES加密过程                                            | 利用SSE/AVX/AVX2向量寄存器，一次处理多个128-bit数据块                                          | ① 提高多块数据处理能力② 适合文件加密、网络数据流场景③ 减少循环开销，提高吞吐量                                   | ① 需要编写向量化代码② 数据排列要求较高③ 对单个数据块性能提升有限                                            |

## SM4:

| 优化方式           | 优化对象       | 核心技术          | 优点            | 缺点           |
|----------------|------------|---------------|---------------|--------------|
| 无优化SM4 (Basic) | SM4轮函数中的S盒 | 按照SM4标准算法直接实现 | 1. 结构简单，代码可读性 | 1. 每轮需要重复执行S |

| 优化方式                                | 优化对象                                    | 核心技术                                                                                                                    | 优点                                                                                                                   | 缺点                                                                                                            |
|-------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| asic)                               | 变换、线性变换L、<br>密钥扩展、32轮迭代<br>计算           | 现，每轮执行：① S盒 $\tau$<br>变换；② 线性扩散L变换<br>；③ 异或运算；④ 轮密<br>钥参与计算                                                             | 强；2. 与SM4标准定义一<br>致，容易验证正确性；3.<br>不需要额外内存和预计算<br>表                                                                   | 盒查询和循环移位；2.<br>大量位运算导致计算开<br>销较高；3. 单线程吞吐<br>率较低，适合作为性能<br>基准而非实际高性能实<br>现                                    |
| T-table优化SM4                        | SM4中的 非线性变<br>换 $\tau$ (S盒) + 线性变<br>换L | 通过预计算方式建立四张<br>查找表：T0、T1、T2、<br>T3，将 $L(\tau(x))$ 合并存储<br>；运行时通过查表代替S<br>盒计算和循环移位                                       | 1. 将复杂计算转换为简单<br>内存访问，提高执行效率<br>；2. 减少运行过程中的循<br>环移位和异或次数；3. 代<br>码改动较小，容易在基础<br>版本上实现；4. 单线程性<br>能提升明显（通常约2~4<br>倍） | 1. 需要额外存储 $4 \times 256$<br>个32位查找表；2. 增加<br>缓存访问压力；3. 查表<br>操作可能产生缓存侧信<br>道风险；4. 不适合对抗<br>高安全要求场景下的侧<br>信道攻击 |
| SIMD AVX2优化<br>SM4                  | SM4加密过程中的<br>数据处理并行性（多<br>个分组同时计算）      | 利用CPU SIMD指令集（<br>AVX2），通过256 bit向<br>量寄存器同时处理多个<br>SM4数据块；使用<br><code>_mm256_xor_si256</code> 等向<br>量指令实现并行异或和数<br>据操作 | 1. 充分利用现代CPU并行<br>计算能力；2. 提高大规模<br>数据加密吞吐率；3. 减少<br>单位数据处理时间；4. 适<br>用于文件加密、网络通信<br>等批量数据场景                          | 1. 依赖硬件支持（需要<br>CPU支持AVX2）；2.<br>编程复杂度较高；3. 对<br>单个128 bit数据块提升<br>有限；4. 需要考虑数据<br>对齐和并行调度问题                  |
| T-table + SIMD<br>组合优化（进一<br>步优化方向） | SM4轮函数计算和数<br>据并行处理                     | 将T-table快速查表与<br>SIMD向量并行结合：利<br>用查表减少单轮计算，再<br>利用AVX2同时处理多个<br>数据块                                                     | 1. 同时降低计算量和提高<br>并行度；2. 理论吞吐率最<br>高；3. 适合高性能密码库<br>实现                                                                | 1. 实现复杂度最高；2.<br>内存访问和SIMD调度需<br>要精细优化；3. 查表仍<br>存在缓存侧信道问题                                                    |

## Gift:

| 优化方式         | 优化对象                                                  | 核心技术                                    | 优点                                                           | 缺点                                                    |
|--------------|-------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------|-------------------------------------------------------|
| 无优化（Basic C） | GIFT整体加密流<br>程，包括S盒替换<br>（SubCells）、比<br>特置换（PermBits | 采用普通C语言实现；S<br>盒使用数组查表；逐字<br>节/逐bit处理数据 | ① 实现简单，代码易理解<br>② 易于调试和验证正确性<br>③ 可作为性能优化的基准<br>版本（Baseline） | ① S盒查表存在内存访<br>问开销② 逐bit置换效<br>率低③ 无数据级并行<br>能力④ 可能受到 |

| 优化方式                        | 优化对象                            | 核心技术                                                                            | 优点                                                                      | 缺点                                                                               |
|-----------------------------|---------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
|                             | ）、轮密钥加（AddRoundKey）             |                                                                                 |                                                                         | Cache侧信道攻击影响                                                                     |
| Bitslice优化                  | 主要优化 S-box(SubCells) 和多个数据块并行处理 | 将多个GIFT状态转化为bit-plane结构；使用逻辑门（AND/XOR/OR/NOT）替代查表；利用64-bit寄存器并行计算多个bit          | ① 消除S盒查表，提高安全性② 避免Cache访问泄露，具有constant-time特点③ 利用CPU字长提高并行度④ 对轻量级密码非常有效 | ① 实现复杂，需要设计Boolean网络② 需要transpose/inverse transpose数据转换③ 单个block处理优势不明显④ 代码可读性下降 |
| SSE/AVX Shuffle优化           | 主要优化 S-box计算、多个数据块并行处理以及状态置换    | 使用SIMD指令；利用128/256 bit寄存器；通过PSHUFEB/VPSHUFEB完成并行查表；利用VPXOR、VPAND、VPERMQ完成并行逻辑运算 | ① 一次处理多个block，提高吞吐率② 充分利用现代CPU SIMD硬件③ 指令级并行，速度最高④ 适合大规模数据加密场景          | ① 对CPU指令集有要求（需要SSSE3/AVX2支持）② 编程复杂度高③ 数据对齐和寄存器管理困难④ 对小数据量加密提升有限                  |
| AVX2 Shuffle + Bitslice混合优化 | S盒和轮函数                          | 将bitslice逻辑与AVX2 SIMD结合；利用__m256i处理256-bit数据；使用__mm256_shuffle_epi8()映射S盒       | ① 综合利用算法级和硬件级优化② 极高吞吐率③ 保持较好的安全性④ 适合服务器、高性能密码库                          | ① 实现难度最高② 调试困难③ 需要针对CPU架构优化④ 移植性较差                                               |

## TWINE:

| 优化方式           | 优化对象                              | 核心技术                                      | 优点                                              | 缺点                                                 |
|----------------|-----------------------------------|-------------------------------------------|-------------------------------------------------|----------------------------------------------------|
| 无优化（Basic C实现） | TWINE整体加密流程，包括密钥扩展、轮函数、S盒替换、置换操作  | 使用标准C语言实现TWINE算法，按照字节/半字节逐步执行36轮Feistel结构 | ①代码结构清晰，容易理解和验证；②与算法描述高度一致，方便调试；③移植性强，不依赖特殊硬件指令 | ①逐字节处理，数据并行能力弱；②大量循环和分支操作导致效率较低；③无法充分利用现代CPU并行计算能力 |
| Bitslice优化     | S盒(Substitution Layer)、轮函数中的非线性变换 | 将多个TWINE数据块按位重新组织，将多个实例的数据位切片存储；利用AND、OR、 | ①利用CPU位级并行能力，一次处理多个数据块；②减少查表操作，提高缓存友好性；③避免传统S   | ①数据布局转换开销较大；②代码复杂度明显增加；③需要重新设计S盒布尔逻辑表达式；④单个数据块加密性  |

| 优化方式                        | 优化对象                | 核心技术                                                                       | 优点                                                                | 缺点                                                       |
|-----------------------------|---------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------|
|                             |                     | XOR等位运算实现S盒                                                                | 盒查表带来的缓存侧信道风险；④适合批量加密场景                                           | 能提升有限                                                    |
| SSE SIMD优化                  | TWINE轮函数中的多个数据块并行处理 | 使用Intel SSE指令集（128 bit SIMD），通过__m128i寄存器同时处理多个数据元素；利用SIMD XOR、AND等指令加速轮函数 | ①一次处理128bit数据，提高吞吐量；②充分利用CPU SIMD执行单元；③相比普通C实现具有明显速度提升；④适合大规模数据加密 | ①依赖CPU支持SSE指令集；②代码可读性下降；③需要处理内存对齐问题；④S盒SIMD化设计复杂；⑤调试难度较高 |
| SSE + Bitslice混合优化（进一步优化方向） | S盒和轮函数              | 将Bitslice的数据布局与SSE寄存器结合，通过多个128bit寄存器并行执行布尔逻辑                              | ①同时利用位级并行和指令级并行；②理论吞吐量最高；③适合密码库高性能实现                              | ①实现复杂度最高；②需要精确设计数据排列；③开发和验证成本较高                          |